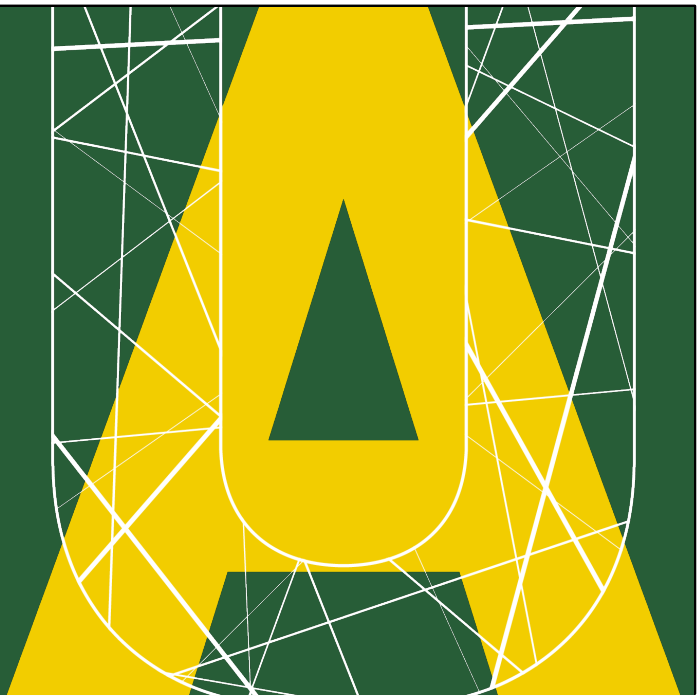
ECE 315

Steven Knudsen, PhD, PEng

Lecture 23 :

- Direct Memory Access (DMA)

UNIVERSITY OF ALBERTA



Last Lecture

- Memory mapping to peripherals

This Lecture

- Direct Memory Access (DMA)

Learning Objectives

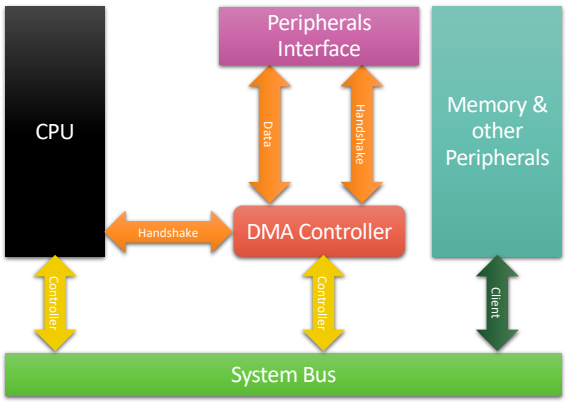
- Explain the motivation for using Direct Memory Access (DMA)
- Describe the architecture and operation of a generic DMA system
- Identify the essential parameters of a DMA transfer
- Describe the steps needed to set up and start a DMA transfer
- Understand the DMA architecture of the RP2040
- Explain the purpose and capabilities of RP2040 DMA channels
- Understand the key parameters in the the RP2040 DMA channel configuration registers

Motivation

- It is very common to move large amounts of data on a computer system
  - Between local devices on the computer bus
  - To and from external devices connected to local devices
- Using the CPU just to move data precludes its use for more complex tasks
- Sometimes the CPU isn't needed, can go to sleep
- A Direct Memory Access (DMA) Controller (DMAC) is a simple peripheral that offloads from the CPU the movement of data

Simple Architecture with DMA

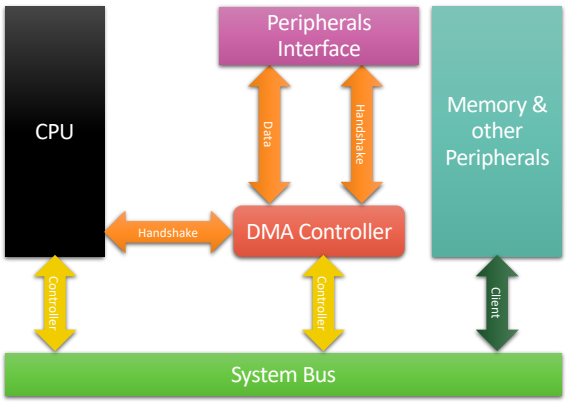
- The DMAC interfaces with the main system bus
  - System Bus is shared by two **Controllers** (**Masters**), CPU and DMAC
  - Other devices on the System Bus are **Clients** (memory, graphics, other peripherals)
- DMAC controlled by CPU
- DMAC can manage select peripherals



UNIVERSITY OF ALBERTA

Types of DMA Transfers (ARM)

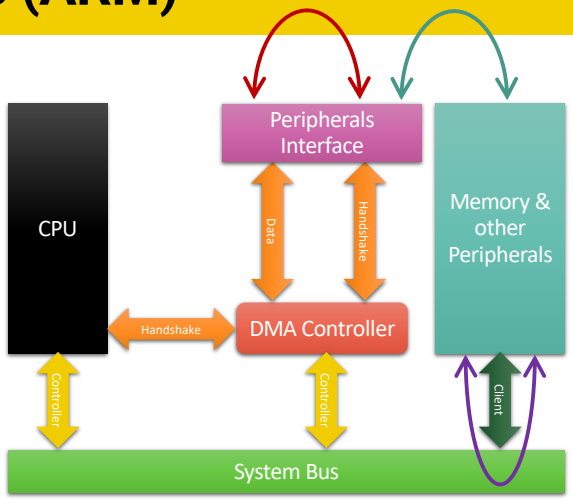
- Memory ↔ Peripheral
- Memory ↔ Memory
- Peripheral ↔ Peripheral



UNIVERSITY OF ALBERTA

Types of DMA Transfers (ARM)

- Memory ↔ Peripheral
- Memory ↔ Memory
- Peripheral ↔ Peripheral



UNIVERSITY OF ALBERTA

DMA Basics

- DMA transfers need
  - Source address
  - Destination address
  - Transfer length
- CPU sets up a DMA transfer by specifying the details and then continues with other tasks
  - The CPU and DMAC operate in parallel
- Interrupts (IRQs) are used in DMA to signal important events:
  - IRQ to signal that an error has occurred during DMA
  - IRQ to signal the end of a DMA transfer

DMA Basics cont'd

- A DMAC is a special-purpose processor
  - Not general like a CPU, not programmable
  - A simpler state machine
- The data transfer algorithm in a DMAC is often hard-coded
  - Behaves like one block move instruction that avoids a large number of fetch-decode-execute cycles for equivalent CPU moves
  - Avoids the overheads associated with moving data using the CPU
- The details of the DMA block move are configured by the CPU by loading values into control registers in the DMAC

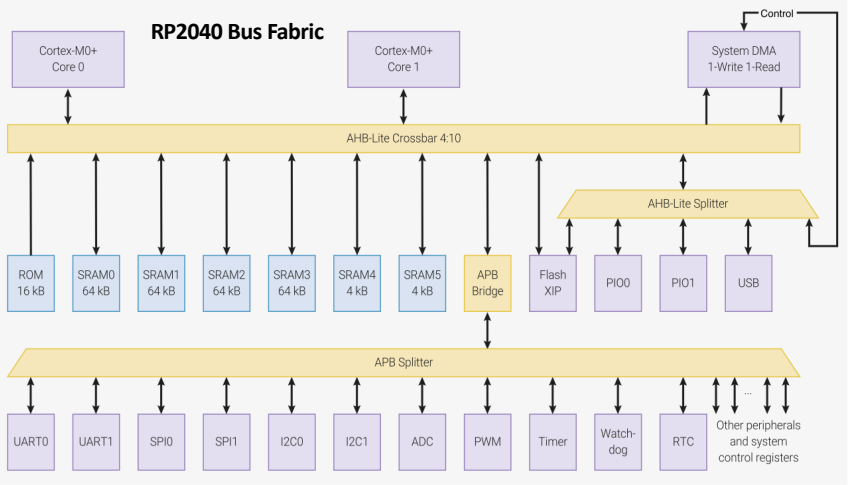
DMA Details - RP2040 Model

- To explain in more depth how DMA works, let's use the RP2040
- The following is derived from the RP2040 data sheet
- First, recall how the Cortex-M0+ CPUs are connected to all the other system components and peripherals

RP2040 Architecture – more detail, bus fabric

AHB is the ***Advanced High-performance Bus*** matrix (crossbar)

- AHB bus decoders match the address to the peripheral
- AHB-Lite system typically has:
  - One AHB master (the Cortex-M0/M0+)
  - Multiple AHB slaves (Flash, SRAM, DMA, AHB-to-APB bridge, etc.)
  - A bus decoder to select which slave responds
- APB = ***Advanced Peripheral Bus*** maps peripherals in memory space



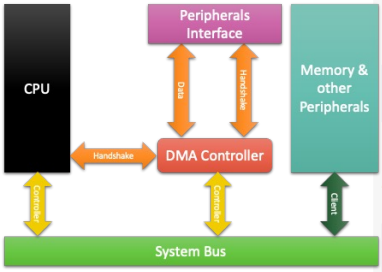
Main reference for the RP2040 material is the RP2040 Datasheet, 2025-02-20

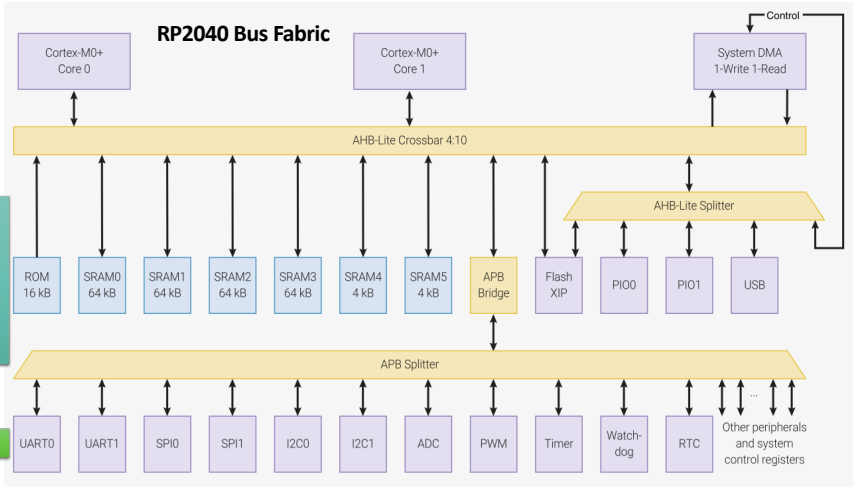
12

RP2040 Architecture – sophisticated DMA

Has the same general form as the generic DMA architecture.

AHB-Lite and APB support access by DMA controller to everything



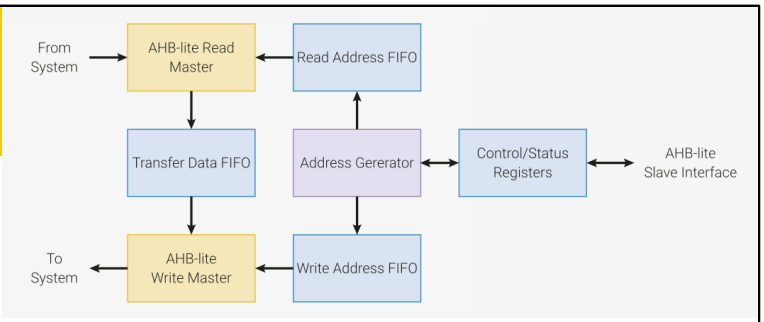


UNIVERSITY OF ALBERTA

Main reference for the RP2040 material is the RP2040 Datasheet, 2025-02-20

13

Simple View

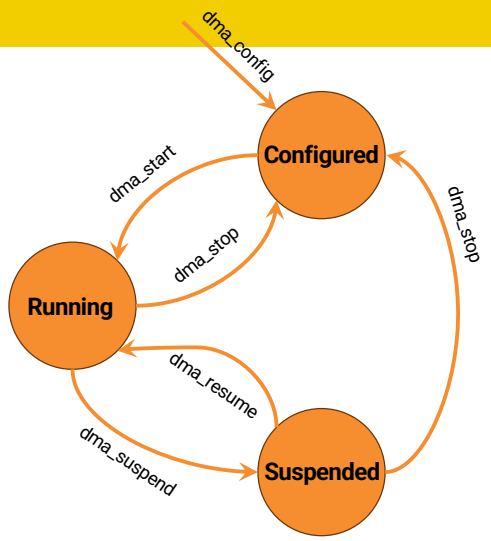


- RP2040 Direct Memory Access (DMA) controller has separate read and write master connections to the bus fabric
- It can perform bulk transfers on behalf of the processors
  - They can do other tasks
  - They can be put to sleep to save power
- DMA can perform one read and one write access every clock cycle
  - 8, 16, or 32 bits

UNIVERSITY OF ALBERTA

RP2040 DMA - Channels

- There are 12 independent channels
  - They are separate state machines
  - They can be combined to support complex data transfers
  - When more than one channel is active, the DMA controller ensure that the bus bandwidth is shared fairly using a round-robin approach
  - Channels can be combined; e.g.,
    - One can configure a second
    - Second invokes the first when it finshes



UNIVERSITY OF ALBERTA

Adapted from https://docs.zephyrproject.org/latest/hardware/peripherals/dma.html

RP2040 DMA – Transfer Modes

- RP2040 DMA supports
  - ***Memory ➔ Peripheral***: a peripheral signals the DMA when it needs more data to transmit. The DMA reads data from an array in RAM or FLASH, and writes to the peripheral's data FIFO†
  - ***Peripheral ➔ Memory***: a peripheral signals the DMA when it has received data. The DMA reads this data from the peripheral's data FIFO and writes it to an array in RAM
  - ***Memory ↔ Memory***: the DMA transfers data between two buffers in RAM, as fast as possible
    - 100s mega-bytes per second (RP2040 clock up to 133 MHz, bus is 32-bits wide)

† FIFO = first-in, first-out buffer

RP2040 DMA – Channel Configuration

- Control/Status registers;
  - `READ_ADDR;` a pointer to the next address to read from
  - `WRITE_ADDR;` a pointer to the next address to write to
  - `TRANS_COUNT;` number of transfers
    - remaining in the current transfer sequence, or
    - number of transfers in the next transfer sequence
  - `CTRL;` configure all other aspects of the channel's behaviour,
    - transfer size, byte, half-word (16 bits), word (32 bits)
    - to enable/disable the channel, and
    - to check for transaction completion
- Registers are updated as the DMA transaction runs. E.g.,
  - `READ_ADDR` and `WRITE_ADDR` are updated by 1, 2, or 4 depending on the transaction data size (8, 16, or 32 bits)

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
31	AHB_ERROR	Logical OR of the READ_ERROR and WRITE_ERROR flags. Channel halts, raises IRQ flag	RO	0x0
30	READ_ERROR	1 indicates read bus error. READ_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
29	WRITE_ERROR	1 indicates write bus error. WRITE_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
28:25	—	*Reserved.*		
24	BUSY	Is 1 when the channel starts a new transfer sequence, 0 when it completes	RO	0x0
23	SNIFF_EN	Enables hardware sniffing engine for this channel.	RW	0x0
22	BSWAP	Apply byte-swap transformation to DMA data	RW	0x0
21	IRQ_QUIET	Channel does not generate IRQs at the end of every transfer block. Instead, an IRQ is raised when NULL is written to a trigger register, indicating the end of a control block chain.	RW	0x0
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

UNIVERSITY OF ALBERTA

19

We'll start with the low bits as they are parameters involved in most or all transactions.

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

UNIVERSITY OF ALBERTA

20

DATA_SIZE and EN are pretty obviously needed.

The defaults are for a data size of byte and the channel is not enabled. The latter makes sense.

A good programmer will always initialize DATA_SIZE, right!?!

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

If we want read from write a block memory device, we set the INCR_READ or INCR_WRITE to 1.

It they are set to zero, the we are reading from writing to the same address. This would be useful if, say, we are using DMA transfers from a sensor, and ADC, or some other device on an SPI or I2C bus.

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

UNIVERSITY OF ALBERTA

22

Ring buffers are super important in any computing system because memory is always finite. In embedded systems this is even more so.

Can you explain how RING_SIZE is related to DATA_SIZE?

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TI 0x3d → TI 0x3e → TI 0x3f → PE ed transfers.	RW	0x00
14:11	CHAIN_TO	When this nel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

RING_SEL is a don't care if RING_SIZE is 0

UNIVERSITY OF ALBERTA

23

Ring buffers are super important in any computing system because memory is always finite. In embedded systems this is even more so.

Can you explain how RING_SIZE is related to DATA_SIZE?

Obviously, RING_SEL is a "don't care" if RING_SIZE is 0

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

UNIVERSITY OF ALBERTA

24

CHAIN_TO is very powerful. The CPU can set up a chain of DMA operations up to 12 long, making for less overhead. This might go forever is the last “chained” DMA channel points back to the first.

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

UNIVERSITY OF ALBERTA

25

The RP2040 runs at 133 MHz (can run faster if overclocked).

Not all peripherals will match that, so it's necessary to "pace" DMA transfers. This is done using interrupts (the peripheral signals it's ready) or timers (the peripheral is "dumb")

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
20:15	TREQ_SEL	Select a Transfer Request signal to pace its data transfer rate. Sources for TREQ signals are internal (TIMERS) or external (DREQ, a Data Request from the system). 0x0 to 0x3a → select DREQ n as TREQ 0x3b → TIMER0: Select Timer 0 as TREQ 0x3c → TIMER1: Select Timer 1 as TREQ 0x3d → TIMER2: Select Timer 2 as TREQ 0x3e → TIMER3: Select Timer 3 as TREQ 0x3f → PERMANENT: Permanent request, for unpaced transfers.	RW	0x00
14:11	CHAIN_TO	When this channel completes, it will trigger the channel indicated by CHAIN_TO.	RW	0x0
10	RING_SEL	If 0, read addresses are wrapped on 1<<RING_SIZE boundary. If 1, write addrs are wrapped.	RW	0x0
9:6	RING_SIZE	Number of low address bits to wrap for ring buffer mode (0=disabled). For n > 0, only lower n bits of the addr change. Addr wraps on a 1 << n byte boundary, facilitating access to naturally-aligned ring buffers.	RW	0x0
5	INCR_WRITE	If 1, the write address increments with each transfer. If 0, always write to same address	RW	0x0
4	INCR_READ	If 1, the read address increments with each transfer. If 0, always read from same address	RW	0x0
3:2	DATA_SIZE	Set the size of each bus transfer (byte/halfword/word); 0x0➔SIZE_BYTE, 0x1➔SIZE_HALFWORD, 0x2➔SIZE_WORD	RW	0x0
1	HIGH_PRIORITY	Gives a channel preferential treatment in issue scheduling	RW	0x0
0	EN	When 1, the channel responds to triggering events, which cause it to become BUSY and start	RW	0x0

Peripheral is “smart”

Peripheral is “dumb”

UNIVERSITY OF ALBERTA

26

The RP2040 runs at 133 MHz (can run faster if overclocked).

Not all peripherals will match that, so it’s necessary to “pace” DMA transfers. This is done using interrupts (the peripheral signals it’s ready) or timers (the peripheral is “dumb”)

RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
31	AHB_ERROR	Logical OR of the READ_ERROR and WRITE_ERROR flags. Channel halts, raises IRQ flag	RO	0x0
30	READ_ERROR	1 indicates read bus error. READ_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
29	WRITE_ERROR	1 indicates write bus error. WRITE_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
28:25	—	*Reserved.*		
24	BUSY	Is 1 when the channel starts a new transfer sequence, 0 when it completes	RO	0x0
23	SNIFF_EN	Enables hardware sniffing engine for this channel.	RW	0x0
22	BSWAP	Apply byte-swap transformation to DMA data	RW	0x0
21	IRQ_QUIET	Channel does not generate IRQs at the end of every transfer block. Instead, an IRQ is raised when NULL is written to a trigger register, indicating the end of a control block chain.	RW	0x0

BUSY is useful if you need to check the state of the transfer, say, to abort it or maybe pause it. Pausing would be tricky, but possible depending on the addresses/peripherals involved.

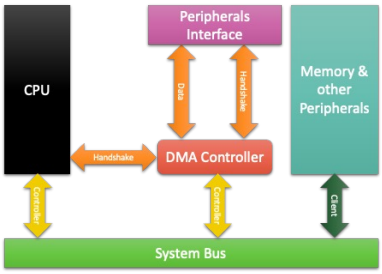
RP2040 DMA – Channel N CTRL Register

Bits	Name	Description	Type	Reset
31	AHB_ERROR	Logical OR of the READ_ERROR and WRITE_ERROR flags. Channel halts, raises IRQ flag	RO	0x0
30	READ_ERROR	1 indicates read bus error. READ_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
29	WRITE_ERROR	1 indicates write bus error. WRITE_ADDR shows the closest address for error. Write 1 to clear	WC	0x0
28:25	—	*Reserved.*		
24	BUSY	Is 1 when the channel starts a new transfer sequence, 0 when it completes	RO	0x0
23	SNIFF_EN	Enables hardware sniffing engine for this channel.	RW	0x0
22	BSWAP	Apply byte-swap transformation to DMA data	RW	0x0
21	IRQ_QUIET	Channel does not generate IRQs at the end of every transfer block. Instead, an IRQ is raised when NULL is written to a trigger register, indicating the end of a control block chain.	RW	0x0

Of course, if there is an error you want to handle it gracefully. These bits provide some clues about errors.

Summary

- Why DMA? ➔ Move large amounts of data without CPU involvement
- DMA Essentials
  - Require *source address*, *destination address*, *transfer length*
  - CPU configures DMA → DMA runs independently
  - Uses IRQs (from peripheral or timer) to signal completion or errors
- Generic DMA Architecture
  - CPU + DMAC are bus masters
  - Peripherals + memory are clients



UNIVERSITY OF ALBERTA

Summary cont'd

- DMA transfers:
  - Memory ↔ Peripheral
  - Memory ↔ Memory
  - Peripheral ↔ Peripheral
- RP2040 DMA Key Features
  - 12 independent DMA channels
  - Separate read and write bus masters
  - Can transfer 8/16/32-bit words, one read + one write each cycle
  - Supports *paced* (TREQ) and *unpaced* transfers
  - Very high throughput (100s MB/s)

Next Lecture

- Quiz #3
- General Q&A

Questions

https://www.tumblr.com/robotatertot

UNIVERSITY OF ALBERTA